Module 2 – C Programming

Lecture 11: Introduction to C

Readings: https://www.tutorialspoint.com/cprogramming/ or
http://www.w3schools.in/c-tutorial/intro/

What is C?

- **Systems programming** language
  - Machine-level concepts (addresses, etc.)
  - Operations close to computer instructions
  - Easier to write than assembly without giving up too much power. Still more error-prone than high-level languages
- Invented in the early 70s
- One of the popular programming languages
  - Operating systems, database engines, game engines, embedded systems, interpreters, etc.
- Small, fast, permissive and portable

What does C look like?

```
/*
 * Program to calculate the factorial of a number
 * Author: Mathieu BV
 */
#include <stdio.h>
int main(void){
    int num, fact;
    fact = 1;
    printf("Enter your number: ");
    scanf("%d", &num); // assumes >= 0 and <= 12
    for (int i = 1 ; i <= num ; i++) {
        fact = fact * i;
    }
    printf("The factorial of %d is %d\n", num, fact);
    return 0; // status code
}
```

How is C run?

- What we saw in the previous slide is called the **source code**
  - Can be written in any text editor: In this course we use vim

```c
#include <stdio.h>
int main(void) {
        puts("hello, world!");
        return 0;
}
```

- The source code is run through a compiler
  - We will use the **gcc** compiler

```
$ gcc hello_world.c -o hello
```

- The compiler checks for errors and creates an executable file
  - This file contains **machine code** (binary) that is understood by the CPU. This is what you will run
  - Compare to Bash, which interprets all commands one by one
  - Compare to Python and Java, which compile to bytecode, an intermediate language that is read by an interpreter

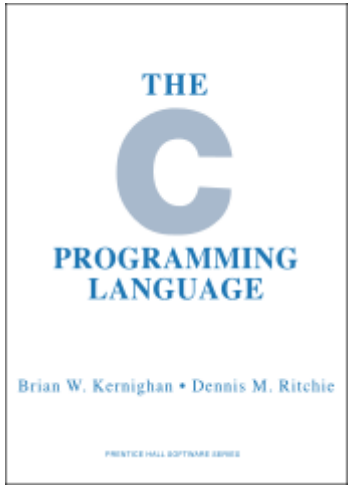
```
00000000 00000000 00000001 00000000 00000010 00000000
01101000 01100101 01101100 01101100 01101111 00101100
00100000 01110111 01101111 01110010 01101100 01100100
00100001 00000000 00000000 00000000 00000001 00011011
00000011 00111011 00110000 00000000 00000000 00000000
00000101 00000000 00000000 00000000 00001100 11110000
```

History of C: Birth

- 1958/60: ALGOL 58/60 (Algorithmic Language)
  - Introduced many ideas including code blocks, function definitions. Too abstract: no input/output, etc.
- 1963: CPL (Combined Programming Language)
  - Inspired by ALGOL 60 but intended for more complex tasks. Too complex, hard to learn, never became popular
- 1967: BCPL (Basic CPL)
  - Basic version of CPL. Small, simple, portable compilers.
- 1969: B
  - Written by Ken Thompson while developing Unix
  - Essentially BCPL with components removed to fit within the memory of PDP-7 computer on which UNIX was developed. Also uses simpler syntax.
  - Compiles to threaded code read by interpreter, very slow
- 1972: C
  - Written by Dennis Ritchie: Started in 1971 as "New B" for PDP-11
  - Adds data types, pointers, and uses compiled machine code
- 1973: Unix is rewritten in C
  - Now you only need to write a C compiler to run Unix on any machine

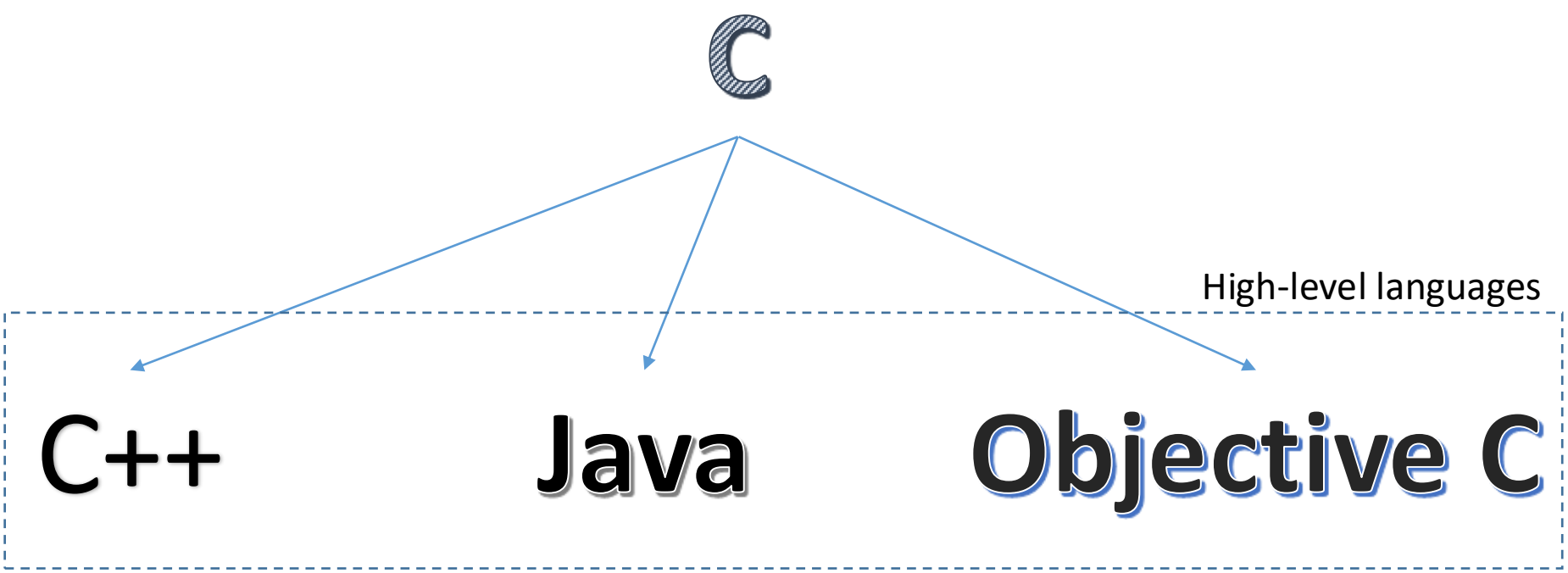
Denis Ritchie
1941 – 2011

History of C: Standardization



- 1978: K&R C
  - The C Programming Language ("The White Book") by Brian Kernighan and Dennis Ritchie is released.
  - De facto standard
- 1989/1990: ANSI/ISO C (C89/C90)
  - Ensures precise and updated description of language and avoids portability issues
- 1999: C99
  - For-scope variables, variable declaration doesn't need to be at start of block, etc.
- Followed by C11, C17, C23 (current)
  - We will mostly stick to "GNU99" in this course, a popular and portable C99 variant with GNU-specific extensions

History of C: Children



1985
Compiled.
Object oriented.
Preserves low-level features.

1995
Highly optimized interpreter.
Object oriented.
Memory managed.
No low-level features.
Cross-platform.

1980
Object-oriented. Uses features from another language called Smalltalk
Standard language for all macOS and iOS applications until 2014 (replaced by Swift, another descendant)

Basic Structure of a C Program

comment

```
/* Hello world program */
#include <stdio.h>
int main(void)
{
    puts("Hello World");
    return 0; // error code
}
```

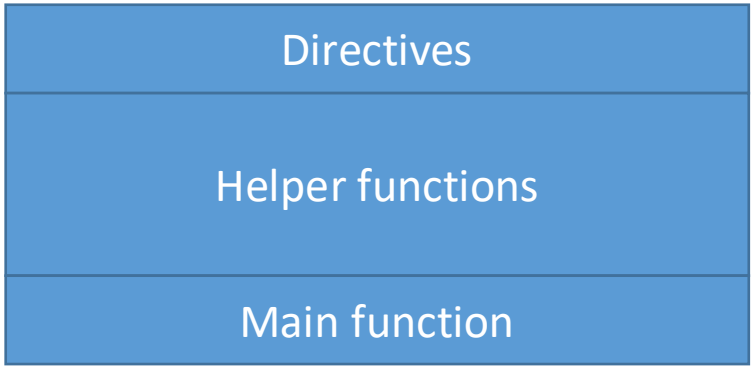
Including libraries is an example of a directive. C is very small and cannot do much without the use of external libraries

main function: Mandatory! The code will start running by calling main(). Must be lowercase.

Statements. Must end with semicolons

(except for compound statements, i.e. blocks)

comment



Recommended layout

Basic Structure of a C Program

STDIO.H is the standard input/output library.
This program uses the following two functions from it:
- puts(), writes string to stdout
- getc(), reads one character from stdin

```
#include <stdio.h>

int main(void)

{
   char c;

   puts("Freshman: ");
   c = getc(stdin);

   if (c == 'Y' || c == 'y'){
    puts("Welcome\n");
   } else {
    puts("Sorry, try again.\n");
   }
   return 0;

}
```

"main" is where an application starts its execution.

Declaring variables.

using variables.

char literals use single quotes

void means that the function does not take any arguments. main() would mean the arguments are unspecified. (Programmers often write main() anyway.)

int is the return type of the function. The main function always returns int since it must return the status code.

string literals use double quotes

gcc

GNU Compiler Collection

- gcc SWITCHES FILES

Switches

- Without a switch the default activity is to merge all the FILES into a single executable file named a.out.
- -o Replace the default a.out file name with your own
  - gcc –o hello helloworld.c
- Will see more switches in later classes

```
Bash-prompt $ gcc –o hello helloworld.c
Bash-prompt $ ./hello
```

Remember that to run programs that are not on your $PATH you must give a path containing a slash. This means adding ./ in front if you are in the same directory as the file.

How to compile and run a C program

```
Bash-prompt $ vim helloworld.c
Bash-prompt $ gcc helloworld.c
Bash-prompt $ ./a.out
```

- We use vim to create our programs (but any program editor can work)
- The GCC compiler is a powerful tool to convert text files into binary machine-code files
- The a.out file is the default binary machine code file name
  - Also known as the Executable file
  - Executable files speak directly with the CPU
- Notice that we execute a.out the same way we executed Bash files, using the ./

- "a.out" is not a clear name. **Use gcc <program.c> -o <name>** to get "name" instead.
  - "-o" for "output file"

puts

Library: stdio.h

Syntax: int puts(constant_string);

Returns: Error code

- >= 0 if no error

Purpose: To print a string to standard out, then newline

Usage:

```
puts("Hello World");        // with one new line
puts("Hello World\n");      // with extra new line
```

Return codes of functions can be ignored if not used.

Escape Characters

\n - New line

\r - Carriage return

\t - Tab

\\ - Backslash

\b - Backspace (non-destructive)

Others…

getc

Library: stdio.h

Syntax: int getc(stdin);

Returns: ASCII code

Purpose: To read a character from standard in

Usage:

```
c = getc(stdin);
```

Notice that this functions does not actually return the character but the ASCII code for that character as an integer number.

This is a low-level feature.

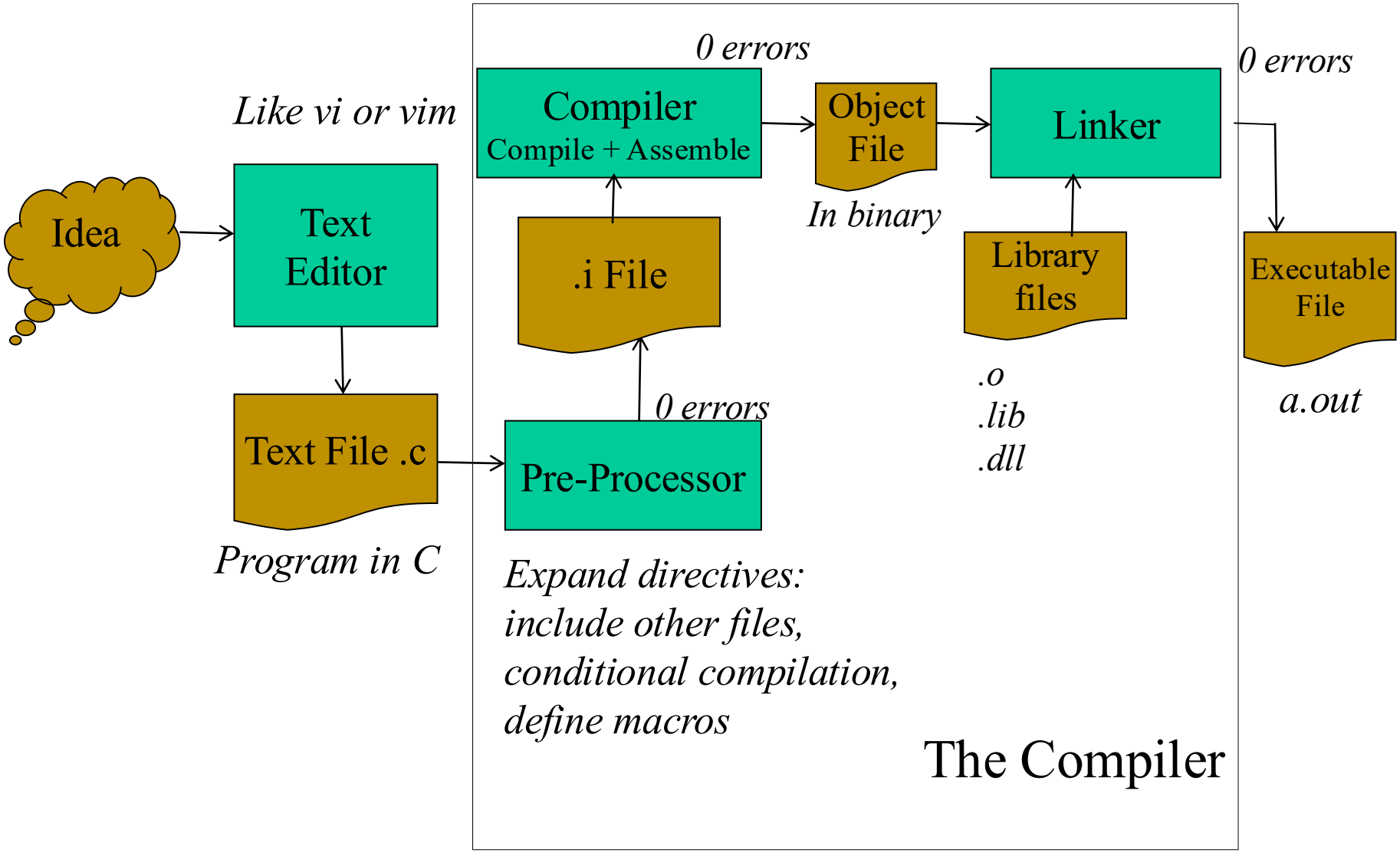
GCC and Errors

Just as you saw in previous lectures, output is sent to standard output and errors are sent to the standard error. Now that we are using C, you will start getting long error messages that might be lost as the screen scrolls.

Solution: gcc helloworld.c 2> textfilename

All output from gcc will be stored in the textfilename file. You can then use vim, more, less, or cat to view the contents.

The C Compilation Process



Note: The compiler sees the .i file (after pre-processor), not your .c file

C Files

- Source Files
  - FILENAME.c .... the program
  - FILENAME.h .... header file (shared code)
- Pre-processed File (Not usually generated for us to see)
  - FILENAME.i
- Assembly Files and Object Files
  - FILENAME.s (Not usually generated for us to see)
  - FILENAME.o
- Executable Files (remember Unix does not care about extensions)
  - FILENAME .... Using the –o switch
  - a.out .... the default executable name

Command Line Processing

```
int main(int argc, char *argv[]) {
   // argc    is like $# +1 (2 arguments => argc == 3)
   // argv[1] is like $1
   // argv[2] is like $2, etc.
   // Note: You will see later that char * in C is like a
   //   string, so argv is an array of strings
   for(int n=0; n<argc; n++) {
       printf("%s\n", argv[n]);
   }
   return 0;
}
```

Resource Credit

These slides were originally prepared by Mathieu Berube-Vallieres

For a course he co-taught with Joseph Vybihal